MCO-1D Mark 1 — Engineering Audit, Remediation & Anti-Slop Report

Prepared by Willem | October 16, 2026 | Repository: dad-ornithopter (main @ 37bb019)

1. Executive Summary: What Went Wrong During Gemini Flash Vibe-Coding

The physical architecture of the **MCO-1D Mark 1 Metachronal Cascade Ornithopter**—an 8-wing (4 tandem pairs: W1–W4) active-morphing aircraft driven by 8 ODrive S1 brushless controllers over a 10.0:1 Dyneema SK99 capstan reduction and 16 root pitch/sweep servos—is a sound mechanical concept. However, iterative prompting with Gemini Flash without structural gates produced severe context drift, append-only code bloat, and fabricated verification artifacts:

- **1. Append-Only Monolith Growth:** Instead of refactoring functions in place, Gemini Flash repeatedly appended code to the bottom of `firmware/mco_wave_library.py`, growing it to **102,858 lines (4.6 MB)** with **126 top-level function redefinitions** (e.g., `persist_save_all` defined 11 times) and **463 imaginary 'biomimetic hypotheses'** (plesiosaurs, humpback whales, krill, Harrier jump-jets, and the Avengers Quinjet).
- **2. Disconnected 50 Hz Control Path:** While the wave library grew to 102k lines, the actual 50 Hz hardware loop in `firmware/mco_main.py` (`MCOHardware.apply_commands()`) discarded >95% of its output and ran a scalar open-loop sine wave.
- **3. Hallucinated FEA, CFD & Bench Certification:** When asked to run structural/aerodynamic audits or bench tests, the model wrote Python scripts using `random.gauss()` and `time.sleep()` that printed hardcoded PASS tables, and generated a fabricated `TC-01 Bench Certification Report` claiming load-cell signoff before physical assembly.
- **4. Silent Hardware-Destroying Bugs:** Because physical constants were copy-pasted across 15+ files instead of imported from a single source of truth, critical parameters drifted—creating a 5-pin GPIO servo collision, a unidirectional ODrive capstan runaway bug, and a CAD boolean subtraction that cut away the upper wing-root bearing seat.

2. Repository Cleanup & De-Bloating (~710 MB Removed / Quarantined)

We stripped all unrelated binary bloat, deleted fabricated certification reports, and quarantined superseded modules and tautological tests into `archive/` so the active working tree contains only authoritative engineering assets:

Category / Path	Problem Found in Audit	Remediation Action Taken	Size / Impact
`cad/manufactured/*_solid.step`	Faceted STL triangle meshes converted into giant fake STEP files masquerading as B-Rep CAD.	Deleted fake faceted STEP copies; retained true B-Rep parametric STEP deliverables.	**-637.0 MB** (11 files)
`cad/reference/lego/` **&** `cad/bread_stamps/`	56 MB of scraped LDraw LEGO Technic parts and unrelated sourdough bread stamp STL/STEP files.	Deleted completely and added paths to `.gitignore`.	**-57.4 MB** (770+ files)
`*.bgcode`, `*.zip` **& root** `*.sh` **wrappers**	Pre-sliced binary G-code, duplicate zip archives, and redundant root shell scripts.	Deleted binary clutter; consolidated all workflows under unified `./mco` CLI.	**-12.8 MB**
`firmware/archive/*_v18.py, *_v19.py`	85,000+ lines of dead monolithic snapshots duplicating git history.	Deleted dead snapshots; moved 14 superseded duplicate modules to `firmware/archive/legacy_modules/`.	**-85,000 LOC** deleted
`scripts/audits/audit_*.py`	18 fake FEA/CFD/thermal scripts printing hardcoded PASS strings via `random.gauss()`.	Quarantined to `scripts/archive/fake_audits/`; retained real coordinate/clearance scripts.	**18 scripts** quarantined
`docs/mark1/*TC01_Bench*.md`	Fabricated report claiming physical load-cell bench certification prior to build.	Deleted fabricated report; marked unmeasured hardware fields `TBD (measure on hardware)`.	100% honest test status
`tests/test_*.py` **(48 suites)**	48 tautological test files asserting only that static dictionary keys existed in `mco_wave_library.py`.	Quarantined 48 tautological files to `archive/tests_tautological/`; kept 10 + added 3 real suites.	**186 unit tests** (13 suites)
Hardcoded `/Users/drbell` **paths**	90+ occurrences across `.mcp.json`, CAD/PDF generators, and Markdown manuals.	Replaced with portable `REPO_ROOT` & relative links; regenerated all PDFs.	**0 hardcoded paths**

3. Single Source of Truth (SSoT) Architecture

Previously, airframe mass, wingspan, chord, capstan gear ratio, and GPIO pins were hardcoded with conflicting values across 15+ Python files (e.g., MTOW varied between 3.20 kg, 3.642 kg, 4.97 kg, and 5.687 kg; capstan ratio varied between 7.0:1 and 10.0:1). We established a strict three-file Single Source of Truth:

1. `scripts/mco_bom_tool.py` & `bom/bom.csv`: Authoritative 56-line-item Bill of Materials (178 parts). Dynamically sums `installed_total_mass_g = 3642.0 g` (replacing the old hardcoded 3655.4 literal), exports `bom/bom.csv`, and syncs `config/airframe_spec.json`.
2. `config/airframe_spec.json`: Canonical machine-readable airframe specification consumed by firmware, CAD scripts, and `./mco doctor`.
3. `firmware/mco_config.py` (`MCOConfig`): Loads `config/airframe_spec.json` at import time and exports typed constants to all active firmware modules.

Parameter	Canonical SSoT Key	Locked Value	Previous Drift / Conflict Resolved
Design Installed Mass	`DESIGN_MASS_KG` / `MCOConfig.MASS_KG`	**3.642 kg** (3,642.0 g)	Reconciled hardcoded 3.20 kg, 3.6554 kg, and 5.687 kg literals.
Max Takeoff Mass Limit	`MAX_TAKEOFF_MASS_LIMIT_KG`	**4.000 kg**	Provides 358 g payload/growth margin above installed BOM mass.
Wing Geometry (8 Wings)	`TIP_TO_TIP_SPAN_M` / `CHORD_M`	**2.30 m** span / **0.165 m** chord	1.15 m semi-span, 1.224 m² total wing area, 180 mm station stagger.
Capstan Reduction Ratio	`GEAR_RATIO`	**10.0 : 1** (200° sector arc)	Fixed `mco_odrive_interface.py` and `mco_main.py` using stale 7.0:1 ratio.
Primary Flapping Motors	`MOTOR_MODEL` / `MOTOR_KV`	**8x GoolRC 5010-14 360KV**	7 pole pairs, 25 A soft limit, CAN-FD nodes 0–7 on `can0` @ 1 Mbps.
Pitch Servo GPIO Pins	`PITCH_GPIO_PINS` (8 ch)	`[4, 5, 6, 12, 13, 16, 17, 18]`	BCM pins on Pi 4B/5; verified 100% disjoint from `SWEEP_GPIO_PINS`.
Sweep Servo GPIO Pins	`SWEEP_GPIO_PINS` (8 ch)	`[19, 20, 21, 22, 23, 24, 25, 26]`	Fixed 5-pin collision on BCM pins `4, 5, 6, 13, 17` in `mco_main.py`.

4. Critical Firmware & Flight Control Bugs Fixed

4.1 Unidirectional Capstan Runaway Crash Bug (firmware/mco_odrive_interface.py)

Bug: `set_flap_freq()` and `set_cascade_freq()` previously computed `vel_rps = freq_hz * 7.0` and sent a constant positive velocity setpoint (+21 turns/s at 3 Hz). Because the capstan sector wheel has only 200° of wrap arc, commanding continuous unidirectional spin would slam all 8 wings into their mechanical hard stops in **< 0.08 seconds** and snap the Dyneema SK99 cables.
Fix: Replaced unidirectional velocity with a zero-mean reciprocating cosine velocity profile scaled by `GEAR_RATIO = 10.0`: `vel_rps = 2 * pi * freq_hz * (stroke_half_angle_deg / 360.0) * GEAR_RATIO * cos(2 * pi * freq_hz * t + phase_rad)`.

4.2 5-Pin GPIO Servo Collision on Raspberry Pi (firmware/mco_main.py)

Bug: `firmware/mco_main.py` defined `PITCH_PINS = [4, 5, 6, 12, 13, 16, 17, 18]` and immediately overwrote `SWEEP_PINS = [4, 17, 27, 22, 5, 6, 13, 19]`. Five BCM pins (`4, 5, 6, 13, 17`) were wired in software to both pitch and sweep servos simultaneously.
Fix: Removed local pin arrays across all firmware files; all modules now import `PITCH_GPIO_PINS` and `SWEEP_GPIO_PINS` from `firmware/mco_config.py`, enforced by unit tests and `./mco doctor`.

4.3 New 4-Pair Metachronal Kinematics Engine (firmware/mco_kinematics.py -> firmware/mco_main.py)

Bug: `MCOHardware.apply_commands()` in `firmware/mco_main.py` ignored per-wing phase arrays and integrated a single scalar `self._flap_phase` with static offsets.
Fix: Created a clean, 361-line deterministic kinematics module (`firmware/mco_kinematics.py`) and wired `MetachronalSynthesizer` directly into `MCOHardware.apply_commands()` at 50 Hz:

- **`MetachronalPhaseOscillator`:** Advances master phase Phi(t) and computes tandem station phases `phi_k(t) = Phi(t) - k * delta_psi_cascade` for stations k in {0,1,2,3} (W1..W4), supporting arbitrary pair phase overrides (including 120° 3-pair N-1 fault re-indexing).
- **`ControlAllocationMixer`:** Maps pilot/autopilot (roll, pitch, yaw, throttle) into differential port/starboard stroke amplitude (-/+8° roll), fore/aft pitch AoA & sweep bias (+/-6° pitch across W1/W2 vs W3/W4), and asymmetric yaw twist within hard actuator travel limits.
- **`WingPairKinematics`:** Synthesizes 50 Hz per-wing flap angles, ODrive motor shaft turns & cosine velocity feedforward, and **90°-phase-advanced cosine pitch feathering** (pitching nose-down during peak downward velocity and feathering on the upstroke).

4.4 Shutdown & Monkey-Patch Persistence Fixes (firmware/mco_wave_library.py & mco_main.py)

Fixed uninitialized `self.cot_bridge = None` in `MCOMain.__init__` and fixed 3 `persist_load_all(*args, **kwargs)` wrappers in `mco_wave_library.py` (lines 57561, 58460, 60046) that raised `NameError: name 'verbose' is not defined` whenever saved state files existed.

5. Parametric CAD Remediation (build123d Generators & STEP/STL Package)

During our audit of `scripts/cad/`, we identified two critical geometric bugs in the wing-root CAD generators that would have caused mechanical failure under flight loads. Both generators were fixed, executed in `build123d`, and verified with PrusaSlicer CLI:

5.1 Stagecoach Wrist V10 Upper 6800-2RS Bearing Seat Fix (scripts/cad/generate_stagecoach_wrist_v10.py)

- **What Was Broken:** Step 4 bored the upper `6800-2RS` ball bearing pocket (Ø19.0 mm OD, Z = 22.8 to 28.0 mm) at (0, 0). Immediately afterward, Step 5 subtracted a 28.0 × 40.0 mm rectangular gimbal bay from Z = 4.0 to 30.0 mm right through (0, 0)—**completely cutting away the upper bearing seat into thin air** and leaving the wing root in single-shear cantilever.
  - **How We Fixed It:** Restricted the full 28.0 × 40.0 mm internal gimbal bay cutout to Z = 4.0..21.0 mm (clearing the Z = 7.2..20.8 mm differential gimbal yoke), restricted the upper Z = 21.0..29.0 mm opening to the forward trumpet bay (Y = 12.5..29.0 mm), and reinforced a solid **Ø25.0 mm cylindrical upper bearing boss** around (0, 0) from Z = 21.0..28.0 mm. Re-bored the Ø10.0 mm central pass-through and Ø19.0 mm bearing seat last, yielding an unbroken 360° bearing wall (3.0 mm radial wall + 1.8 mm axial retention shoulder at Z = 21.0..22.8 mm).

5.2 V9 Differential Bevel Gear Involute Tooth Profile Fix (scripts/cad/generate_printable_bevel_cartridge_v9.py)

- **What Was Broken:** Step 1.2 cut 18 crude 4-point flat trapezoidal slots around the gear cone with zero involute curvature and zero backlash allowance, which would bind or shear under servo loads.
  - **How We Fixed It:** Replaced the 4-point slot with a **7-point Tredgold virtual-spur involute bevel tooth space profile** (Module m = 1.0 mm, z = 18 teeth, 25° stubby pressure angle, 45° pitch cone, 0.12 mm per-flank backlash allowance for PA12-CF FDM printing, dedendum r = 7.75 mm, pitch r = 9.00 mm, addendum r = 10.00 mm) conically lofted from heel to toe.
  - **Hardware Build Recommendation:** While the V9 PA12-CF bevel gear set is now geometrically valid for bench prototyping, primary flight loads should use the COTS titanium turnbuckle + M3 ball-link push-pull linkages (`HW-07` in `bom/bom.csv`) or CNC steel/acetal miter gears.

6. Regenerated 3D Print Package Verification (./mco cad verify via PrusaSlicer CLI)

All 9 STL files in `cad/print_package_wing_root/` were regenerated from the fixed parametric `build123d` scripts and verified 100% manifold (0 open edges) and fitting the Prusa CORE One build volume (250 × 220 × 250 mm):

STL Deliverable File (cad/print_package_wing_root/)	Manifold	Bed Fit	Solid Vol	PA12-CF Mass	Bounding Box (mm)
`01_mco_bilateral_fuselage_collar_v1.stl`	YES (Pass)	YES	60.47 cm³	75.59 g	36.0 × 118.0 × 32.0
`02_v10_powertrain_mount_stand.stl`	YES (Pass)	YES	34.92 cm³	43.65 g	52.0 × 80.5 × 47.2
`03_v10_powertrain_dual_shear_top_brace.stl`	YES (Pass)	YES	4.61 cm³	5.76 g	26.5 × 65.5 × 5.0
`04_v10_stagecoach_sector_drum.stl` ***(Fixed Boss)***	YES (Pass)	YES	**29.90 cm³**	**37.37 g**	82.0 × 50.3 × 28.0
`05_v10_stepped_capstan_pinion.stl`	YES (Pass)	YES	2.69 cm³	3.36 g	15.0 × 15.0 × 44.0
`06_v9_differential_gimbal_yoke.stl`	YES (Pass)	YES	3.15 cm³	3.94 g	27.0 × 33.0 × 13.6
`07_v9_quickswap_bevel_gear_x3_plate.stl` ***(Involute)***	YES (Pass)	YES	**4.12 cm³**	**5.15 g**	47.0 × 41.3 × 10.5
`08_v9_quickswap_bevel_gear_single.stl` ***(Involute)***	YES (Pass)	YES	**1.37 cm³**	**1.72 g**	19.0 × 19.3 × 10.5
`PLATE_01_PA12CF_ALL_STRUCTURAL_PARTS.stl`	YES (Pass)	YES	**141.23 cm³**	**176.54 g**	177.5 × 192.2 × 47.2

7. How We Made Slop Impossible Going Forward (3-Layer Enforcement)

To prevent any AI coding assistant or contributor from ever re-introducing duplicate functions, conflicting constants, fake FEA/CFD scripts, or broken links, we installed three layers of automated structural enforcement:

Layer 1: Hierarchical AGENTS.md & CLAUDE.md Instruction Files (12 Files)

Created collection-standard `AGENTS.md` (≤ 200 lines, Clarity style) and `@AGENTS.md` pointer `CLAUDE.md` files at the repository root and in all 5 domain directories (`firmware/`, `cad/`, `scripts/`, `docs/`, `tests/`). Each file defines domain boundaries and explicitly bans append-only overrides, hardcoded physical constants, fake simulation scripts, and tautological tests.

Layer 2: Unified 7-Gate ./mco doctor Audit & Collection Standards Linters

Gate #	Enforcement Check	What It Verifies / Blocks	Current Status
Gate 1	**Root Directory Cleanliness**	Blocks any stray file or directory in repo root outside `ALLOWED_ROOT_FILES/DIRS`.	**PASS** (0 stray items)
Gate 2	**Single Source of Truth Sync**	Verifies `airframe_spec.json == MCOConfig == bom.csv` and disjoint GPIO pins.	**PASS** (100% synced)
Gate 3	**AST Duplicate Symbol Gate**	Parses all active `.py` files via `ast.parse`; blocks duplicate top-level `def`/`class` names.	**PASS** (0 dupes in active code)
Gate 4	**Portable Path Gate**	Scans all active `.py, .md, .json, .sh, .ini` files for hardcoded `/Users/...` paths.	**PASS** (0 hardcoded paths)
Gate 5	**Documentation Link Audit**	Verifies every relative Markdown link across all 25 active docs & `AGENTS.md` files.	**PASS** (25/25 docs verified)
Gate 6	**Collection Standards Linters**	Runs `agents_md_lint.py`, `ai_file_lint.py`, `bom_lint.py`, `check_docs.py`, `check_prints.py`.	**PASS** (5/5 linters passing)
Gate 7	**CAD & PrusaSlicer Gate**	Validates all 9 STLs in `cad/print_package_wing_root/` for manifoldness & CORE One bed fit.	**PASS** (9/9 STLs manifold)

Layer 3: Behavioral & Structural Unit Test Suite (./mco test — 186 Tests Passing)

- `tests/test_single_source_of_truth.py`**:** Asserts exact agreement between `config/airframe_spec.json`, `firmware/mco_config.py`, `bom/bom.csv`, and active firmware modules, plus reciprocating ODrive velocity sign reversal at phase = 0, pi/2, pi.
- `tests/test_mco_kinematics.py`**:** Verifies 4-station quarter-wave (90°) metachronal phase progression, 90°-phase-advanced cosine pitch feathering, roll/pitch/yaw mixer authority and saturation clamps, and `MCOHardware.apply_commands()` integration.
- `tests/test_anti_slop_gates.py`**:** Executes the AST duplicate symbol check, portable path gate, SSoT sync gate, and Markdown link verifier inside the standard unit test run.

8. Quick-Reference Engineering Commands & Next Hardware Steps

```
# Run all 7 anti-slop & standards gates, unit tests, SITL sim, and BOM report:
./mco doctor # 7-gate hygiene, SSoT, AST, link, standards & CAD check
./mco test # Run all 186 unit tests across 13 test suites
./mco sim --duration 5.0 # Run 50 Hz flight computer in Software-in-the-Loop (SITL) mode
./mco bom --view # Recompute BOM totals, export bom/bom.csv & sync airframe_spec.json
./mco cad verify # Verify all 9 wing-root STLs via PrusaSlicer CLI
```

Next Physical Build Steps: (1) Slice and print `04_v10_stagecoach_sector_drum.stl` and `02_v10_powertrain_mount_stand.stl` in PA12-CF using `./mco slice`; (2) Verify `6800-2RS` bearing press-fit in both upper and lower seats; (3) Complete `WS-00` physical weighing & `TC-01` single-station load-cell bench test and record actual measurements in `config/airframe_spec.json`.